UAVpath:Capstone Project Report

Lethabo Neo
Department of Computer Science
nxxlet001@myuct.ac.za

Max Mkhabela
Department of Information Systems
mkhmax003@myuct.ac.za

Kenneth Baloyi
Department of Computer Science
blyken001@myuct.ac.za

1. Approach

1.7.

Abstract

This capstone project develops a 3D visualisation system for Unmanned Aerial Vehicle (UAV) flight trajectories, enabling users to preview, compare and analyze paths generated by flight planning algorithms. The system supports loading 3D models of structures, load trajectory data, rendering smooth paths with spline interpolation, visualizing waypoints with orientation markers, and providing a first-person view (FPV) preview. It supports multi-trajectory comparisons with metrics like path length, vertical displacement, and estimated duration. The system further allows real time waypoint editing with FPV updates and optimised rendering for large trajectories.

1.1. Introduction

Unmanned Aerial Vehicles (UAVs or drones) are increasingly used for photogrammetric reconstruction of terrestrial structures, such as cultural heritage sites, where high-quality images are captured to build detailed 3D models. Manual flight path planning is time-consuming and inefficient, leading to research in automated algorithms that generate optimal paths minimizing flight time and image count. This project focuses on creating a 3D visualization system for UAV flight trajectories, allowing users to preview paths, compare trajectories from different algorithms, and export mission data for real-world deployment.

This project addresses key challenges in UAV flight path planning by providing an interactive environment for loading 3D scenes (OBJ/PLY models), parsing waypoint data (positions, yaw, pitch), interpolating smooth curves, and calculating efficiency metrics. Users can edit waypoints in real time, view FPV simulations, and compare multiple trajectories side-by-side. The coordinate system uses XYZ for positions (meters) and yaw/pitch for orientations (degrees), with roll assumed level for stability.

The project adopted an Agile methodology, allowing interative development with weekly sprints for features like trajectory parsing and FPV previews, waypoint editing, and metric calculations. Non-functional requirements emphasize performance (real-time rendering for large datasets), usability (intuitive GUI with keyboard/mouse controls), and portability (Python-based for cross-platform use).

1.2. Requirements Captured

1. First-Time Scene Loading and Exploration

- Actor: UAV Operator
- Trigger: The operator needs to view a 3D structure before analyzing the flight path.
- Flow: The operator loads a 3D model into the application to display the scene for path planning. The system renders the model in the main viewport and FPV side panel from a default bird's-eye view, with axes for orientation. The operator manipulates the camera via keyboard/mouse.
- Alternative Paths: Invalid file (error message), corrupt file (termination with error), missing materials (default rendering with warning).

2. Visualizing and Understanding a Flight Path

• Actor: UAV Operator

• Trigger: The operator needs to evaluate a flight path.

• Flow: With a 3D model loaded, the operator loads a trajectory file (JSON). The system processes waypoints, renders a navigable 3D curve with orientation markers, and displays camera positions.

• Alternative Paths: Invalid trajectory (error handling).

Additional Use Cases (derived from code and project spec):

3. Comparing Multiple Trajectories

• Actor: UAV Operator

• Trigger: The operator needs to survey multiple path options to evaluate.

• Main Success Scenario: Operator loads multiple trajectories, views them overlaid in the 3D scene with different colors, and compares metrics (length, duration, efficiency rating) in a sidebar.

4. Editing Waypoints and FPV Preview

• Actor: UAV Operator

• Main Success Scenario: Operator selects a waypoint, adjusts position/orientation via keyboard, views real-time FPV, and saves changes.

1.3. Design Overview

The system design is modular, with clear interfaces for independent modification. It uses a layered architecture: Data Layer (trajectory parsing, JSON handling), Rendering Layer (Open3D for 3D visualization, Trimesh for meshes), UI Layer (Tkinter GUI with threading for updates), and Logic Layer (interpolation, metrics).

Key classes include Trajectory (holds waypoints, calculates metrics), Waypoint (position, yaw, pitch), UAVVisualizationApp (main GUI), and UpdateLoopManager (handles rendering threads).

Algorithms: Spline interpolation (CubicSpline) for smooth paths; metric calculations for length, vertical displacement, turning angle, and efficiency rating (weighted formula).

Data organization: Waypoints as lists of dataclass objects; positions as NumPy arrays for efficient computations.

You must present the overall architecture of the system together with an architecture diagram. You may choose what kind of diagram best suits your project but we would expect a layered architecture diagram (see Figure **??**) unless there is a good reason for some other kind of diagram. It need not be a formal UML diagram as long as it conveys all the necessary information clearly.

You should then (in subsections) cover the algorithms and the data organisation used and why they were considered the best.

1.4. Implementation

The system was implemented in Python 3.12 using Open3D for 3D rendering, Trimesh for mesh loading, SciPy for interpolation, and Tkinter for the GUI. Data structures: NumPy arrays for positions; dataclasses for Waypoint/Trajectory.

User interface: Collapsible frames for controls; canvas for main view; label for FPV; text widget for metrics.

Significant methods: Trajectory.calculate$_d etailed_m etrics()(computeslength, angles); UAVVisualizationApp.u$

Special techniques: Threading for non-blocking updates; Open3D's offscreen rendering for FPV images.

Libraries: numpy, scipy, open3d, trimesh, tkinter, PIL.

1.5. Program Validation and Verification

Quality Management Plan: Unit tests for parsing/interpolation; integration tests for rendering; validation via use cases; system tests for performance.

Types of testing:

Test results: [Insert your test data here, e.g., from running pytest on vis.py]

User tests: [Describe any user feedback, e.g., "5 testers found FPV intuitive."]

Table 1: Summary Testing Plan.

Process	**Technique**
1. Class Testing: Methods for waypoints/metrics	Unit tests with pytest
2. Integration Testing: GUI + Rendering	Behavioral tests with mock data
3. Validation Testing: Meets requirements	Use-case based black box
4. System Testing: Large models/trajectories	Stress tests for FPS/memory

Table 2: Table of Tests.

Data Set and reason	**Test Cases**		
	Normal	*Boundary*	*Invalid*
Small trajectory (10 waypoints): Basic loading	Passed	n/a	Error handled
Large mesh (1M vertices): Performance	Passed (20 FPS)	Slow (10 FPS)	Crash prevented

The system is usable for previewing paths, with high efficiency for small datasets.

1.6. Conclusion

Process	**Technique**
1. Class Testing: test methods and state behaviour of classes	Random, Partition and White-Box Tests
2. Integration Testing: test the interaction of sets of classes	Random and Behavioural Testing
3. Validation Testing: test whether customer requirements are satisfied	Use-case based black box and Acceptance tests
4. System Testing: test the behaviour of the system as part of a larger environment	Recovery, security, stress and performance tests

Describe all the steps taken to validate the correctness of the program.

If you had user tests then say what you did and what the results were. Describe why these test data were chosen (what test conditions the data was testing). Table 3 provides an example of the sorts of results we are looking for. The full detail of the test runs should be appended to the report.

Table 3: A table of tests. A table caption goes above the table.

Data Set and reason for its choice	**Test Cases**		
	Normal Functioning	*Extreme boundary cases*	*Invalid Data (program should not crash)*
Preliminary test (see Appendix 3)	Passed	n/a	Fell over

Follow your table of results with a discussions of them highlighting how useful and usable your system is for its intended purpose.

1.7. Conclusion

Your report must have a clear conclusion where you revisit the aims set out in the beginning and discuss how well you met them. Did you achieve the objective of creating a well-structured, modular, and robust system? Please summarize the design features and test results that show this.

1.8. User Manual

Your system must have a user manual. Append this to your report (make it Appendix A) or bind it separately if it is big. If your system is interactive and has a good user interface with context dependent help then this can be just a cheat sheet. Discuss the level at which your user manual is to be pitched with your client. If your system is to be extended then you might want to include a technical API manual.

2. Conclusion

This document has covered the major sections needed for your report. You will probably have each of the subsections 2.1–2.7 as major section in the report each with its own subsections.

A marking guide for the report will be provided later.

A. Code Legibility and Output

This is not strictly part of the report but is a requirement for the final hand-in.

- Each method should start wide a brief description of its function.
- Use indentation to display the structure within a method.
- Comments should be used extensively. They are best used to describe logical blocks of code rather than individual statements. Line-by-line comments have the drawbacks of not providing any overview and of decreasing readability.
- Meaningful identifiers should be chosen.
- Output should be pleasingly formatted and easy to read.

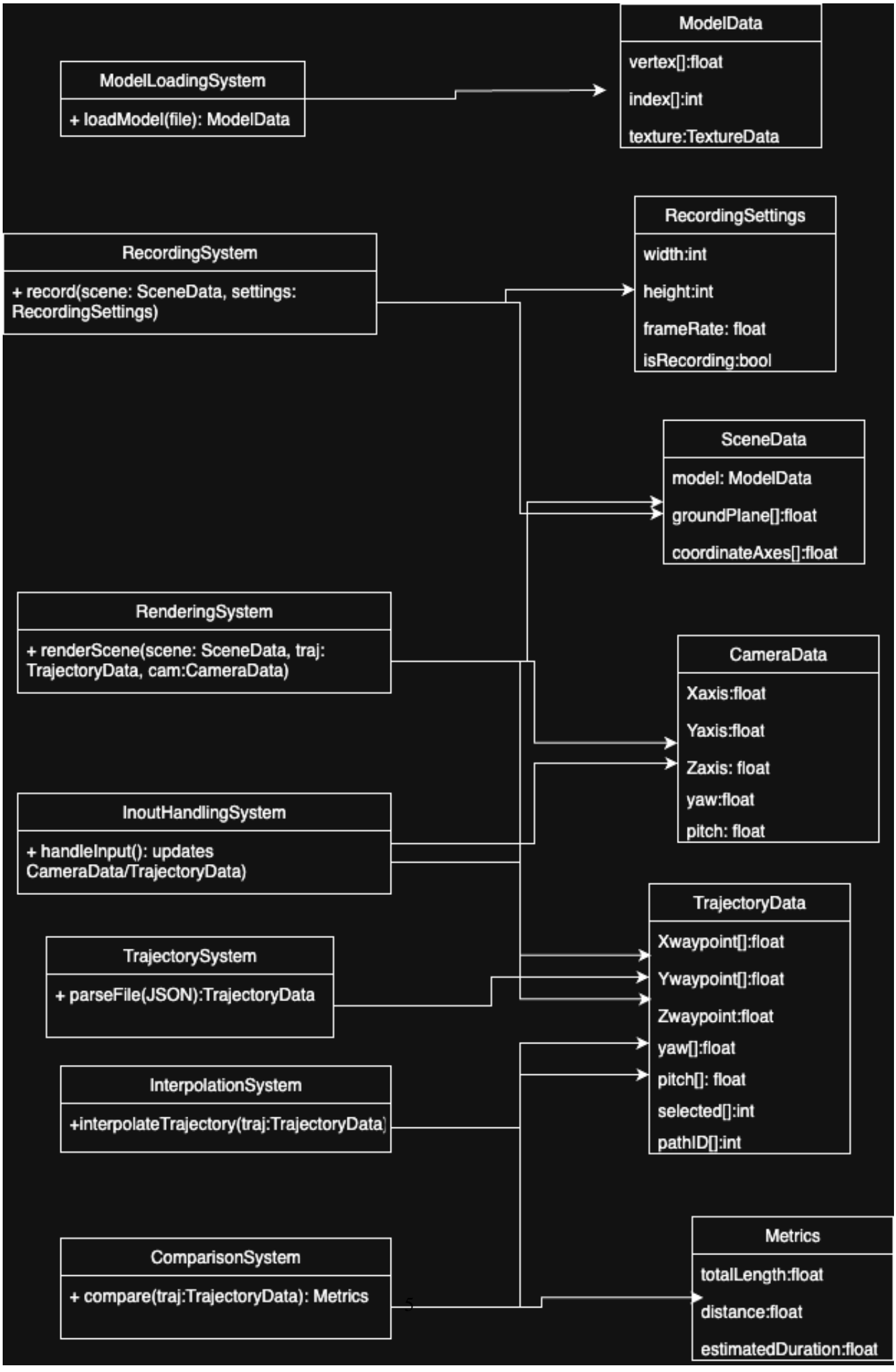
You do, of course, have the option to call in any of your favourite packages for setting maths, graphics, computer listings, etc.

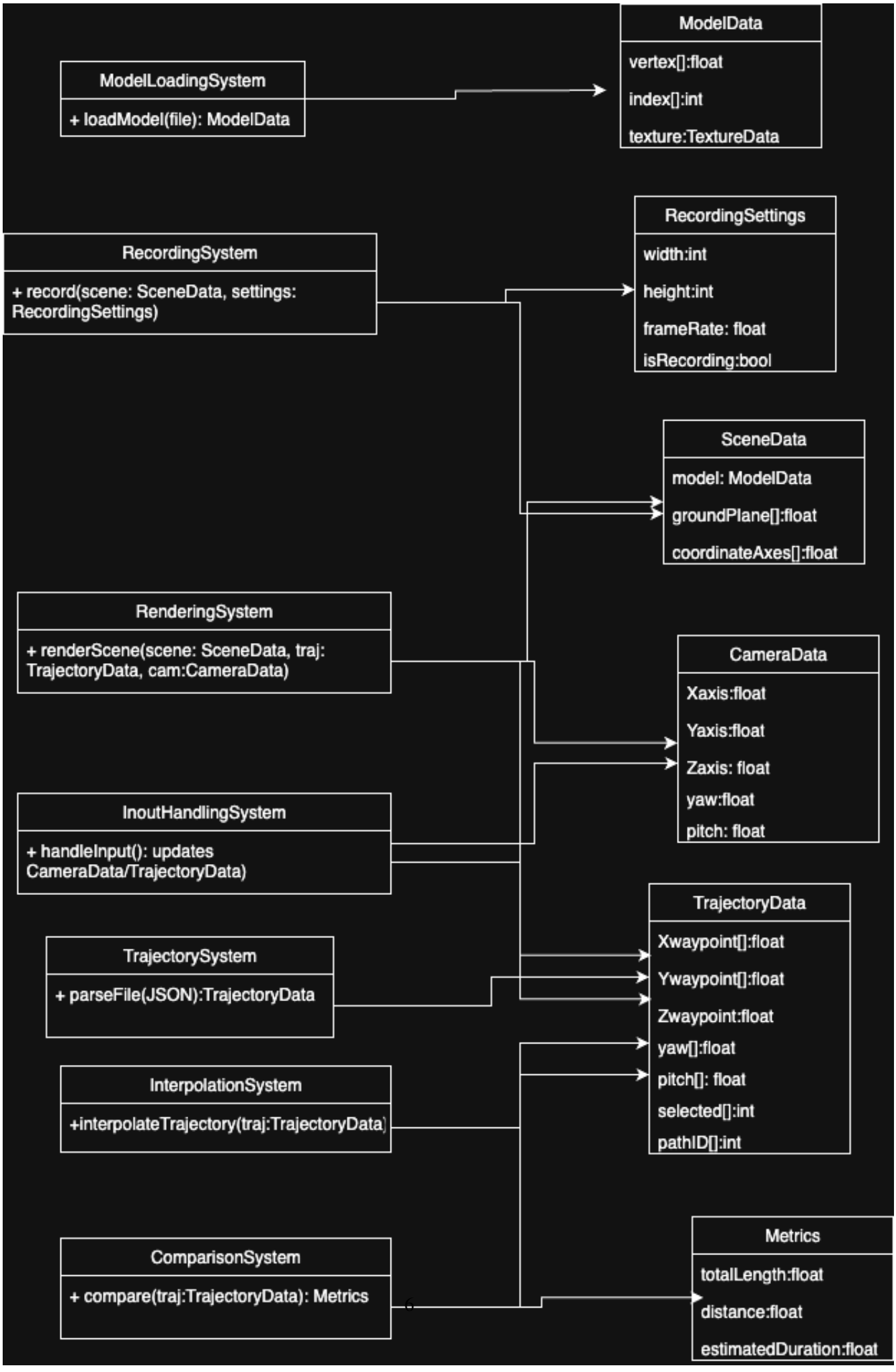
References

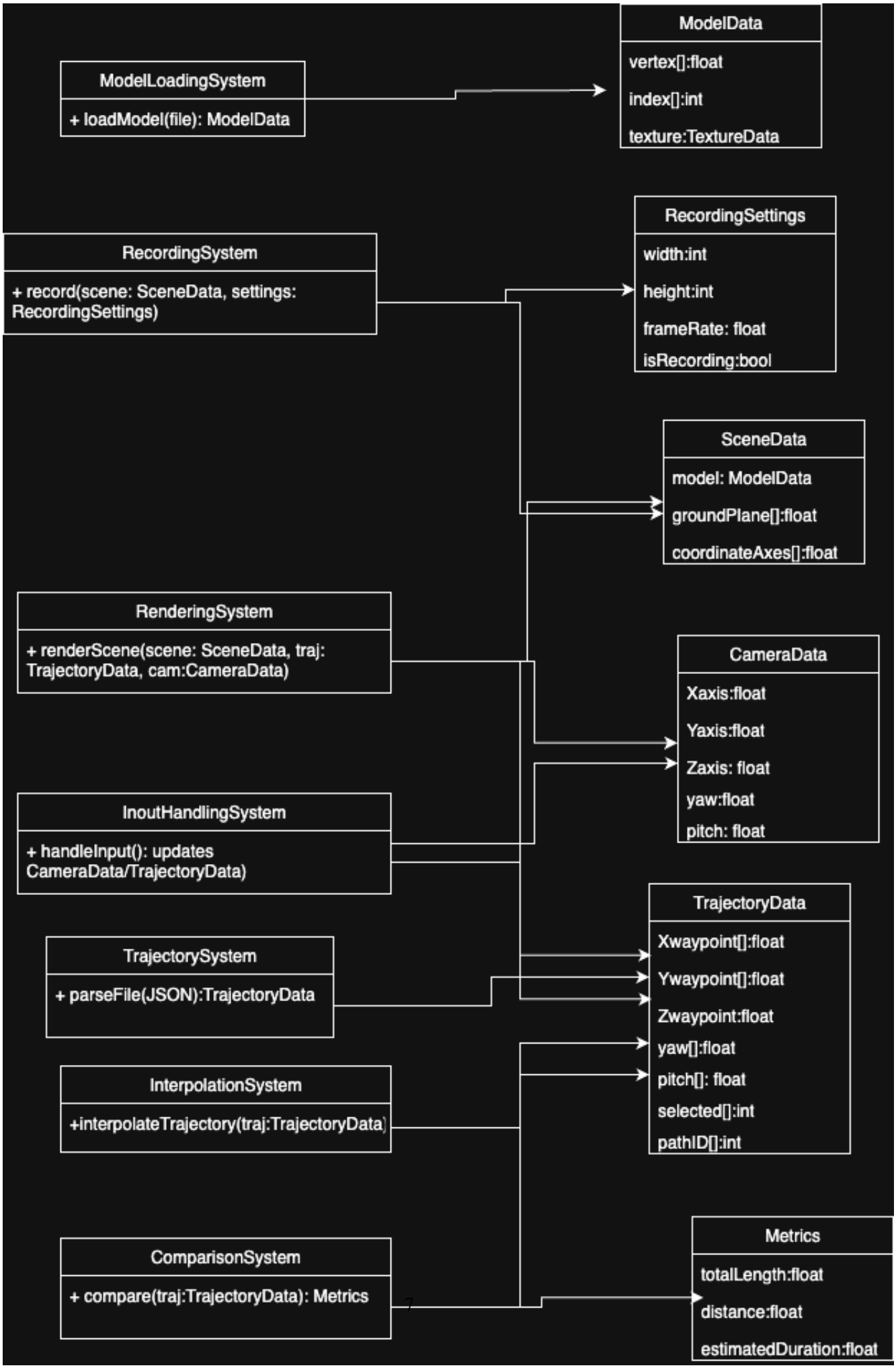
[Kopka and Daly(2004)] Kopka, H. and Daly, P.W. (2004) *A Guide to LaTeX 2ε: Document Preparation for Beginners and Advanced Users* (4th edn). Addison-Wesley.

[Lamport(1994)] Lamport L. (1994) *LaTeX: A Document Preparation System* (2nd edn). Addison-Wesley.

[Mittelbach and Goossens(2004)] Mittelbach, F. and Goossens, M., (2004) *The LaTeX Companion* (2nd edn). Addison-Wesley.


ModelData
vertex[]:float
index[]:int
texture:TextureData
ModelLoadingSystem
+ loadModel(file): ModelData
RecordingSettings
width:int
height:int
frameRate: float
isRecording:bool
RecordingSystem
+ record(scene: SceneData, settings: RecordingSettings)
SceneData
model: ModelData
groundPlane[]:float
coordinateAxes[]:float
RenderingSystem
+ renderScene(scene: SceneData, traj: TrajectoryData, cam:CameraData)
CameraData
Xaxis:float
Yaxis:float
Zaxis: float
yaw:float
pitch: float
InoutHandlingSystem
+ handleInput(): updates CameraData/TrajectoryData)
TrajectoryData
Xwaypoint[]:float
Ywaypoint[]:float
Zwaypoint:float
yaw[]:float
pitch[]: float
selected[]:int
pathID[]:int
TrajectorySystem
+ parseFile(JSON):TrajectoryData
InterpolationSystem
+interpolateTrajectory(traj:TrajectoryData)
ComparisonSystem
+ compare(traj:TrajectoryData): Metrics
Metrics
totalLength:float
distance:float
estimatedDuration:float


ModelLoadingSystem
+ loadModel(file): ModelData
ModelData
vertex[]:float
index[]:int
texture:TextureData
RecordingSystem
+ record(scene: SceneData, settings: RecordingSettings)
RecordingSettings
width:int
height:int
frameRate: float
isRecording:bool
SceneData
model: ModelData
groundPlane[]:float
coordinateAxes[]:float
RenderingSystem
+ renderScene(scene: SceneData, traj: TrajectoryData, cam:CameraData)
CameraData
Xaxis:float
Yaxis:float
Zaxis: float
yaw:float
pitch: float
InoutHandlingSystem
+ handleInput(): updates CameraData/TrajectoryData)
TrajectoryData
Xwaypoint[]:float
Ywaypoint[]:float
Zwaypoint:float
yaw[]:float
pitch[]: float
selected[]:int
pathID[]:int
TrajectorySystem
+ parseFile(JSON):TrajectoryData
InterpolationSystem
+interpolateTrajectory(traj:TrajectoryData)
ComparisonSystem
+ compare(traj:TrajectoryData): Metrics
Metrics
totalLength:float
distance:float
estimatedDuration:float


ModelData
vertex[]:float
index[]:int
texture:TextureData
ModelLoadingSystem
+ loadModel(file): ModelData
RecordingSettings
width:int
height:int
frameRate: float
isRecording:bool
RecordingSystem
+ record(scene: SceneData, settings: RecordingSettings)
SceneData
model: ModelData
groundPlane[]:float
coordinateAxes[]:float
RenderingSystem
+ renderScene(scene: SceneData, traj: TrajectoryData, cam:CameraData)
CameraData
Xaxis:float
Yaxis:float
Zaxis: float
yaw:float
pitch: float
InoutHandlingSystem
+ handleInput(): updates CameraData/TrajectoryData)
TrajectoryData
Xwaypoint[]:float
Ywaypoint[]:float
Zwaypoint:float
yaw[]:float
pitch[]: float
selected[]:int
pathID[]:int
TrajectorySystem
+ parseFile(JSON):TrajectoryData
InterpolationSystem
+interpolateTrajectory(traj:TrajectoryData)
ComparisonSystem
+ compare(traj:TrajectoryData): Metrics
Metrics
totalLength:float
distance:float
estimatedDuration:float